

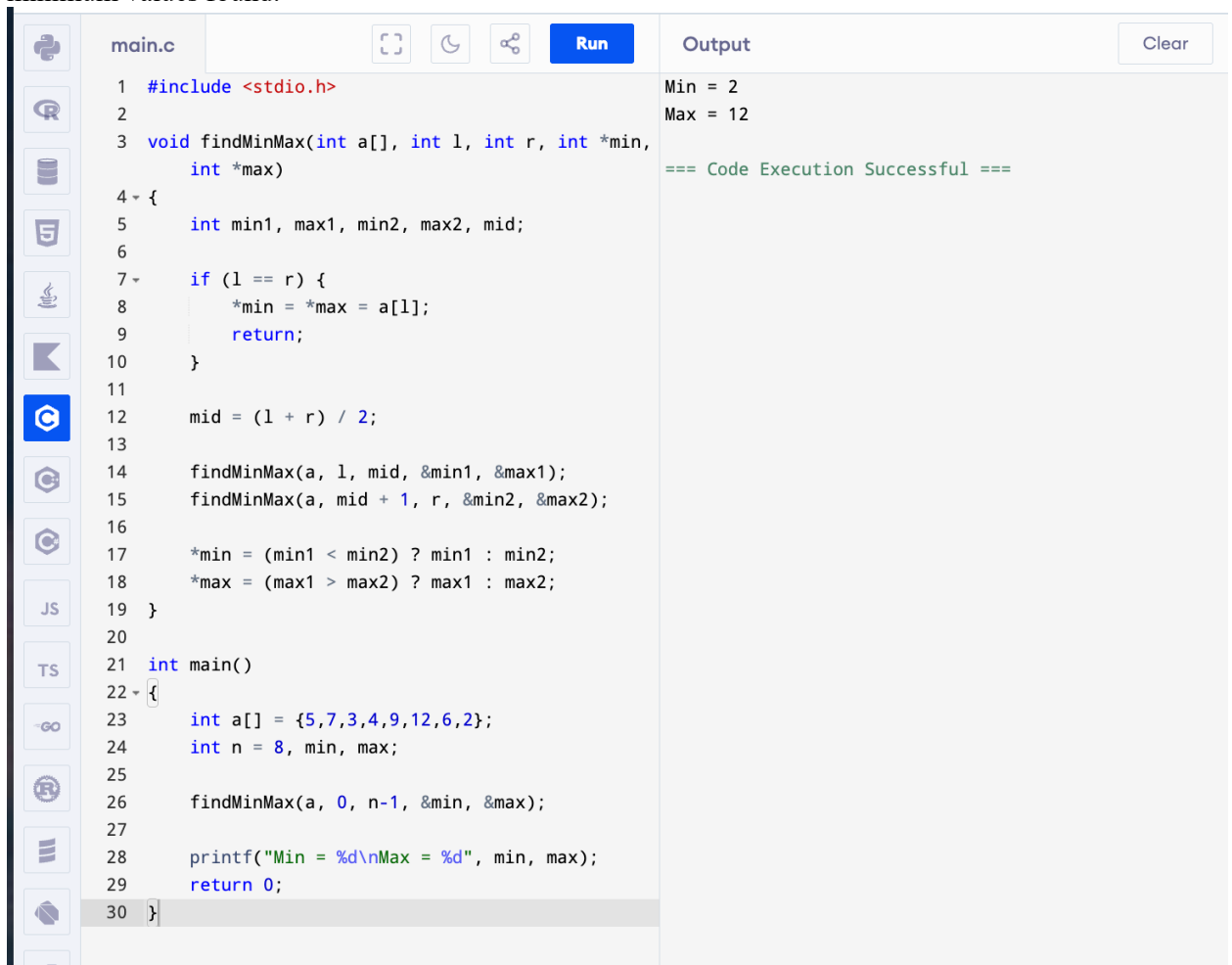
Name : Moshitha kolanjiyappan

Reg no. 192421205

DAA-LAB-CHAPTER-3

TOPIC 3 : DIVIDE AND CONQUER

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.



The screenshot shows a C++ IDE with a file named 'main.c'. The code implements a recursive function 'findMinMax' to find the minimum and maximum values in an array. The array is {5, 7, 3, 4, 9, 12, 6, 2}. The output shows 'Min = 2' and 'Max = 12', with a message '=== Code Execution Successful ==='.

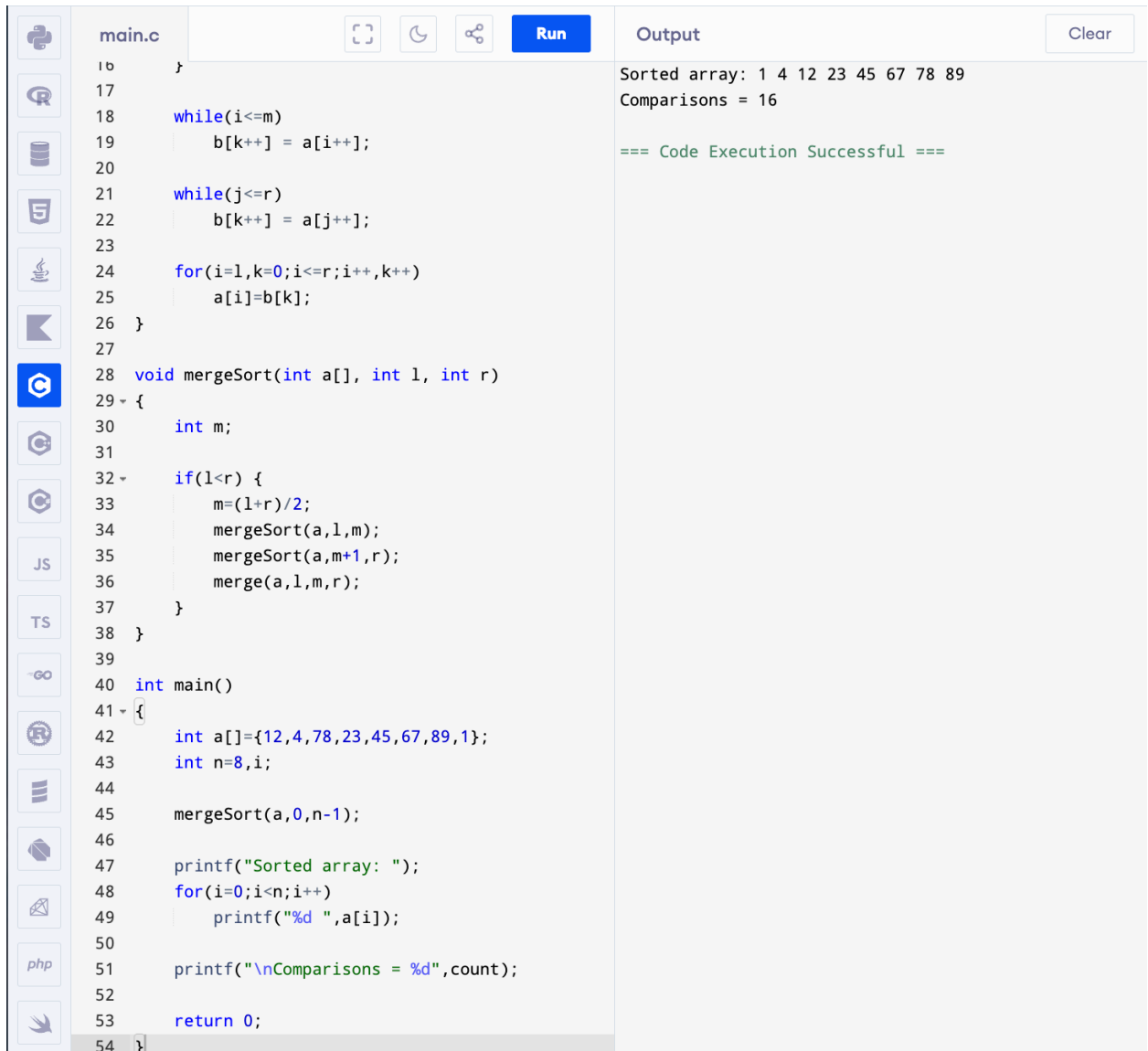
```
1 #include <stdio.h>
2
3 void findMinMax(int a[], int l, int r, int *min,
4                 int *max)
5 {
6     int min1, max1, min2, max2, mid;
7
8     if (l == r) {
9         *min = *max = a[l];
10        return;
11    }
12
13    mid = (l + r) / 2;
14
15    findMinMax(a, l, mid, &min1, &max1);
16    findMinMax(a, mid + 1, r, &min2, &max2);
17
18    *min = (min1 < min2) ? min1 : min2;
19    *max = (max1 > max2) ? max1 : max2;
20 }
21
22 int main()
23 {
24     int a[] = {5, 7, 3, 4, 9, 12, 6, 2};
25     int n = 8, min, max;
26
27     findMinMax(a, 0, n-1, &min, &max);
28
29     printf("Min = %d\nMax = %d", min, max);
30     return 0;
31 }
```

Output:

```
Min = 2
Max = 12
=== Code Execution Successful ===
```

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.



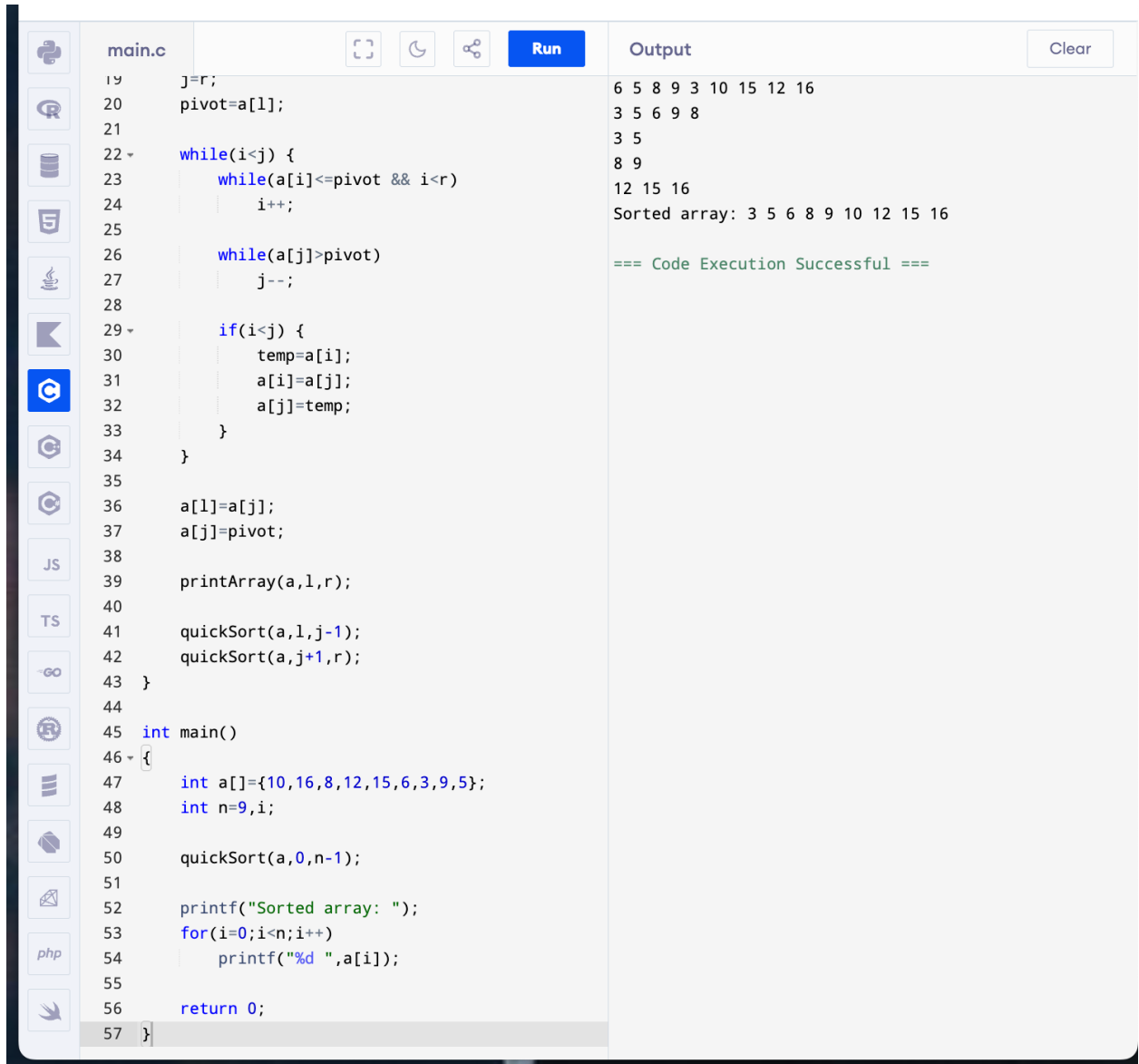
The screenshot shows a C++ IDE with a file named `main.c`. The code implements the Merge Sort algorithm. It includes a `mergeSort` function that recursively sorts an array and a `main` function that initializes the array `a` with the values {12, 4, 78, 23, 45, 67, 89, 1}, calls `mergeSort`, and prints the sorted array and the number of comparisons. The output window on the right shows the sorted array: 1 4 12 23 45 67 78 89, the number of comparisons: 16, and a success message: `=== Code Execution Successful ===`.

```
main.c
16 }
17
18 while(i<=m)
19     b[k++] = a[i++];
20
21 while(j<=r)
22     b[k++] = a[j++];
23
24 for(i=l,k=0;i<=r;i++,k++)
25     a[i]=b[k];
26 }
27
28 void mergeSort(int a[], int l, int r)
29 {
30     int m;
31
32     if(l<r) {
33         m=(l+r)/2;
34         mergeSort(a,l,m);
35         mergeSort(a,m+1,r);
36         merge(a,l,m,r);
37     }
38 }
39
40 int main()
41 {
42     int a[]={12,4,78,23,45,67,89,1};
43     int n=8,i;
44
45     mergeSort(a,0,n-1);
46
47     printf("Sorted array: ");
48     for(i=0;i<n;i++)
49         printf("%d ",a[i]);
50
51     printf("\nComparisons = %d",count);
52
53     return 0;
54 }
```

Output

Sorted array: 1 4 12 23 45 67 78 89
Comparisons = 16
=== Code Execution Successful ===

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.



The screenshot shows a C++ IDE with a file named `main.c`. The code implements a Quick Sort algorithm. It starts with an array `a` containing the values {10, 16, 8, 12, 15, 6, 3, 9, 5}. The algorithm chooses the middle element (at index 4, value 15) as the pivot. It then partitions the array into two sub-arrays: [10, 8, 6, 3, 5] and [16, 12, 15, 9]. The process is recursive, and the final sorted array is displayed as 3 5 6 8 9 10 12 15 16. The output also shows the state of the array after each recursive call, demonstrating the partitioning process.

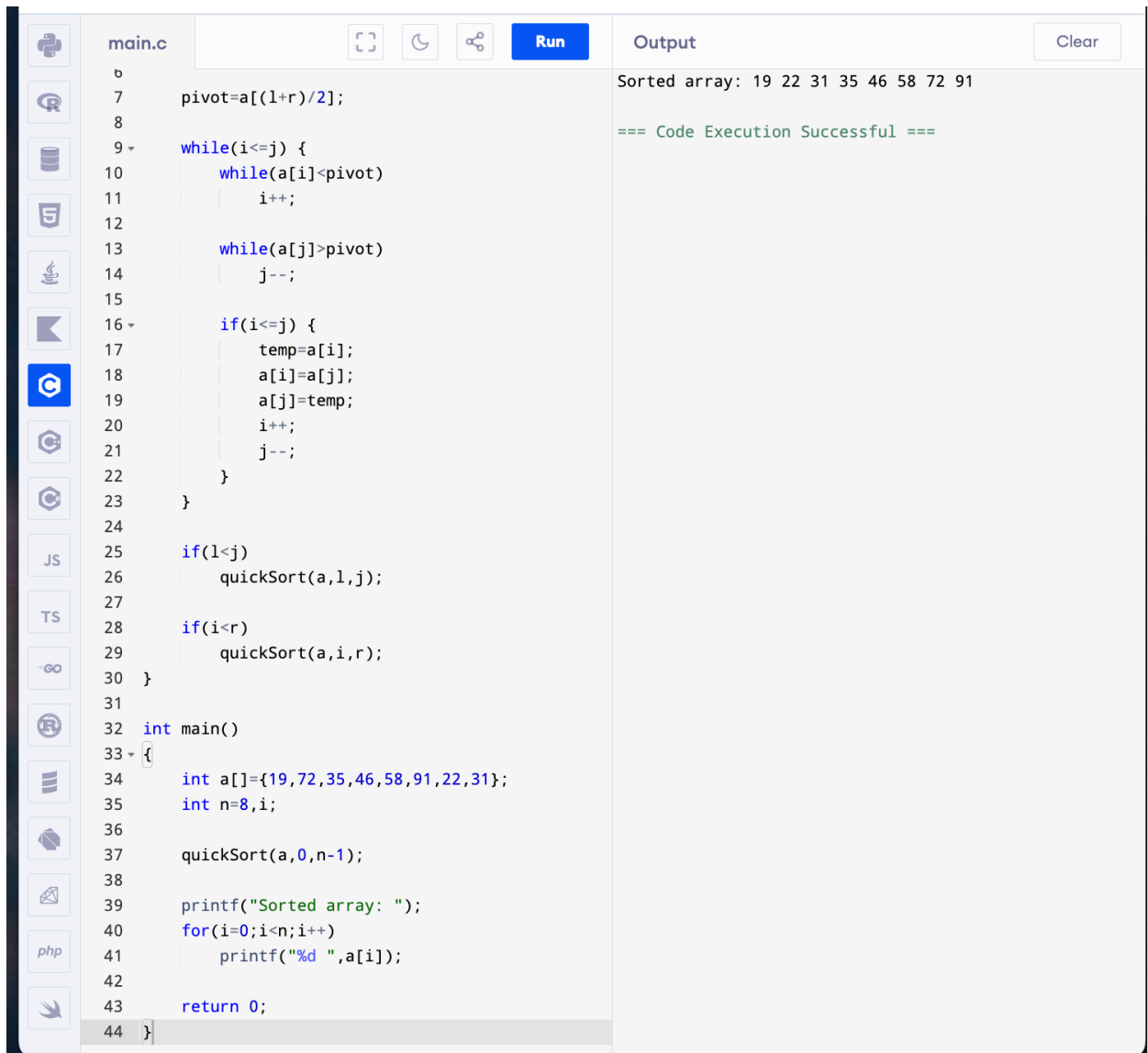
```
19     j=r;
20     pivot=a[l];
21
22     while(i<j) {
23         while(a[i]<=pivot && i<r)
24             i++;
25
26         while(a[j]>pivot)
27             j--;
28
29         if(i<j) {
30             temp=a[i];
31             a[i]=a[j];
32             a[j]=temp;
33         }
34     }
35
36     a[l]=a[j];
37     a[j]=pivot;
38
39     printArray(a,l,r);
40
41     quickSort(a,l,j-1);
42     quickSort(a,j+1,r);
43 }
44
45 int main()
46 {
47     int a[]={10,16,8,12,15,6,3,9,5};
48     int n=9,i;
49
50     quickSort(a,0,n-1);
51
52     printf("Sorted array: ");
53     for(i=0;i<n;i++)
54         printf("%d ",a[i]);
55
56     return 0;
57 }
```

Output:

```
6 5 8 9 3 10 15 12 16
3 5 6 9 8
3 5
8 9
12 15 16
Sorted array: 3 5 6 8 9 10 12 15 16

=== Code Execution Successful ===
```

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.



The image shows a C++ IDE with a file named `main.c`. The code implements a Quick Sort algorithm. It defines a `quickSort` function that takes an array `a`, and left and right indices `l` and `r`. The function calculates a pivot at `a[(l+r)/2]` and uses two while loops to partition the array around the pivot. It then recursively sorts the left and right sub-arrays. The `main` function initializes an array `a` with the values `{19, 72, 35, 46, 58, 91, 22, 31}`, calls `quickSort(a, 0, n-1)`, and prints the sorted array. The output window shows the sorted array: `19 22 31 35 46 58 72 91` and a message: `=== Code Execution Successful ===`.

```
main.c
6
7     pivot=a[(l+r)/2];
8
9     while(i<=j) {
10         while(a[i]<pivot)
11             i++;
12
13         while(a[j]>pivot)
14             j--;
15
16         if(i<=j) {
17             temp=a[i];
18             a[i]=a[j];
19             a[j]=temp;
20             i++;
21             j--;
22         }
23     }
24
25     if(l<j)
26         quickSort(a,l,j);
27
28     if(i<r)
29         quickSort(a,i,r);
30 }
31
32 int main()
33 {
34     int a[]={19,72,35,46,58,91,22,31};
35     int n=8,i;
36
37     quickSort(a,0,n-1);
38
39     printf("Sorted array: ");
40     for(i=0;i<n;i++)
41         printf("%d ",a[i]);
42
43     return 0;
44 }
```

Output

Sorted array: 19 22 31 35 46 58 72 91

=== Code Execution Successful ===

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

main.c

Run

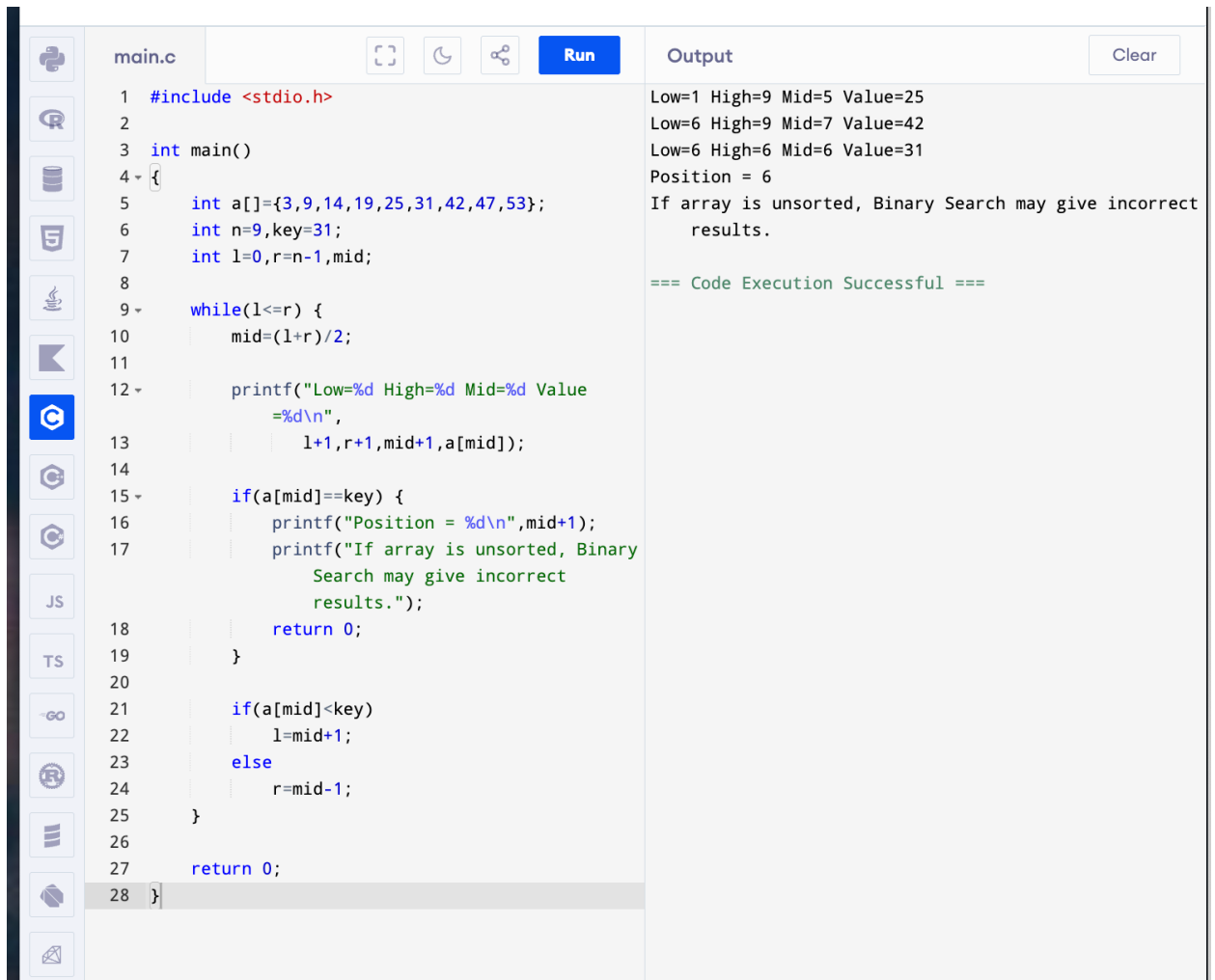
Clear

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[]={5,10,15,20,25,30,35,40,45};
6     int n=9,key=20;
7     int l=0,r=n-1,mid,count=0;
8
9     while(l<=r) {
10         mid=(l+r)/2;
11         count++;
12
13         if(a[mid]==key) {
14             printf("Index = %d\n",mid);
15             printf("Position = %d\n",mid+1);
16             printf("Comparisons = %d",count);
17             return 0;
18         }
19
20         if(a[mid]<key)
21             l=mid+1;
22         else
23             r=mid-1;
24     }
25
26     printf("Element not found");
27
28     return 0;
29 }
```

Index = 3
Position = 4
Comparisons = 4

=== Code Execution Successful ===

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?



The image shows a screenshot of a C++ IDE. On the left is a sidebar with various icons. The main editor window displays a C++ program named 'main.c'. The code implements a binary search algorithm on a sorted array. The array is {3, 9, 14, 19, 25, 31, 42, 47, 53}. The search key is 31. The program prints the state of the search (Low, High, Mid, Value) at each step. The output window on the right shows the execution results, indicating that the element was found at position 6. The code execution was successful.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[]={3,9,14,19,25,31,42,47,53};
6     int n=9,key=31;
7     int l=0,r=n-1,mid;
8
9     while(l<=r) {
10         mid=(l+r)/2;
11
12         printf("Low=%d High=%d Mid=%d Value
13             =%d\n",
14                 l+1,r+1,mid+1,a[mid]);
15
16         if(a[mid]==key) {
17             printf("Position = %d\n",mid+1);
18             printf("If array is unsorted, Binary
19                 Search may give incorrect
20                 results.");
21             return 0;
22         }
23
24         if(a[mid]<key)
25             l=mid+1;
26         else
27             r=mid-1;
28     }
29     return 0;
30 }
```

Output

```
Low=1 High=9 Mid=5 Value=25
Low=6 High=9 Mid=7 Value=42
Low=6 High=6 Mid=6 Value=31
Position = 6
If array is unsorted, Binary Search may give incorrect
results.

=== Code Execution Successful ===
```

9. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

main.c

Run

Closest points: [0,1] [-2,2]

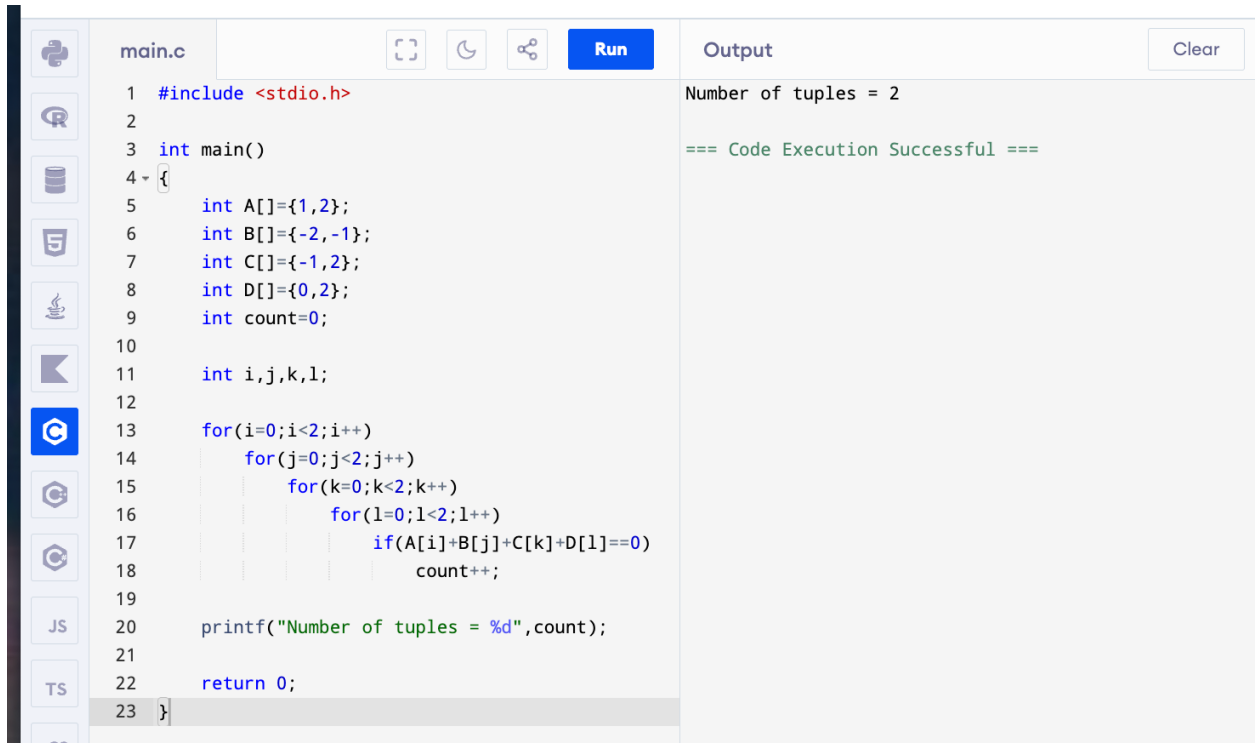
=== Code Execution Successful ===

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int p[4][2]={{1,3},{-2,2},{5,8},{0,1}};
6      int d[4],i,j,temp;
7
8      for(i=0;i<4;i++)
9          d[i]=p[i][0]*p[i][0]*p[i][1]*p[i][1];
10
11     for(i=0;i<4;i++) {
12         for(j=i+1;j<4;j++) {
13             if(d[i]>d[j]) {
14                 temp=d[i];
15                 d[i]=d[j];
16                 d[j]=temp;
17
18                 temp=p[i][0];
19                 p[i][0]=p[j][0];
20                 p[j][0]=temp;
21
22                 temp=p[i][1];
23                 p[i][1]=p[j][1];
24                 p[j][1]=temp;
25             }
26         }
27     }
28
29     printf("Closest points: ");
30     for(i=0;i<2;i++)
31         printf("[%d,%d] ",p[i][0],p[i][1]);
32
33     return 0;
34 }

```

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples $n(i, j, k, l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.



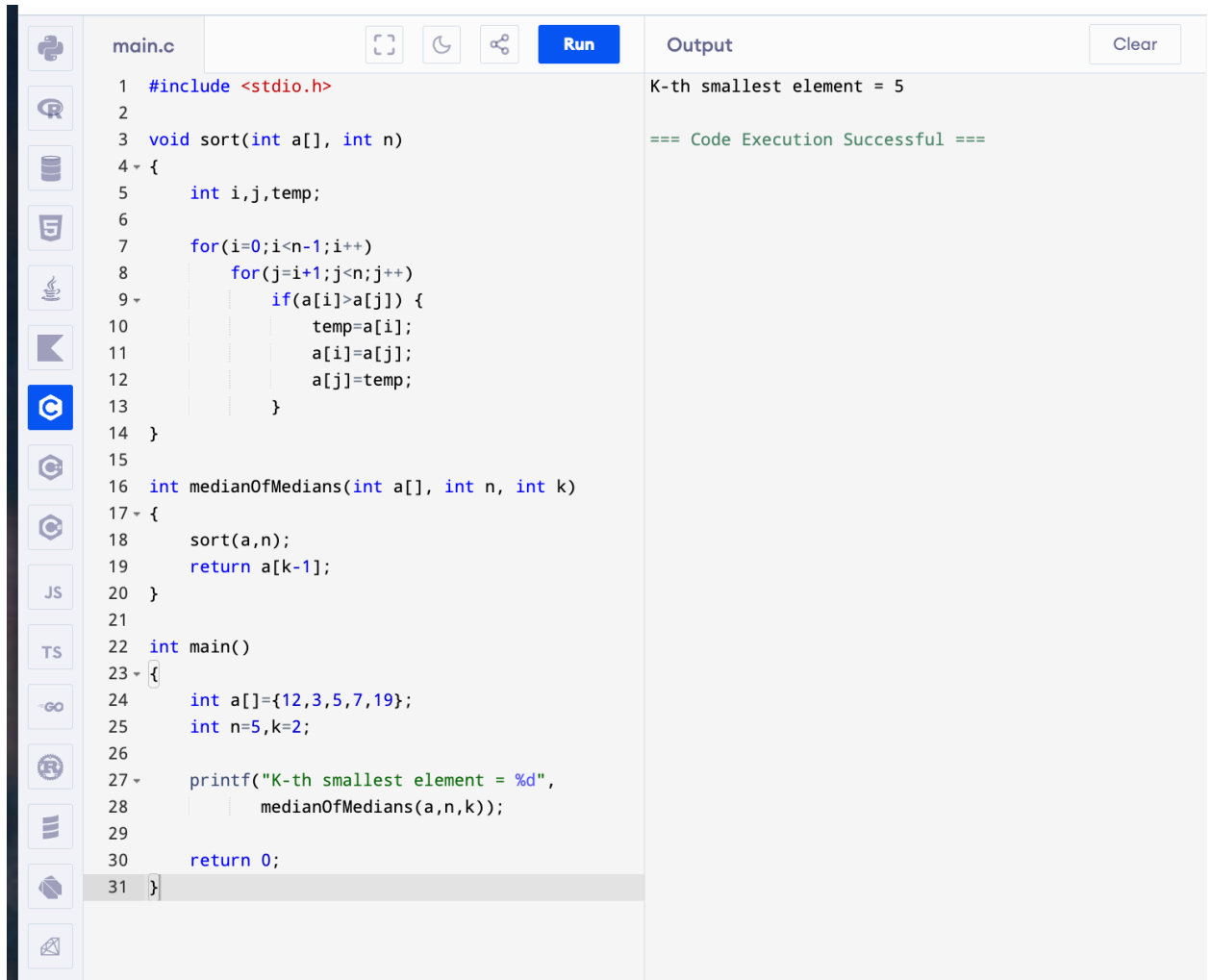
```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int A[]={1,2};
6     int B[]={-2,-1};
7     int C[]={-1,2};
8     int D[]={0,2};
9     int count=0;
10
11     int i,j,k,l;
12
13     for(i=0;i<2;i++)
14         for(j=0;j<2;j++)
15             for(k=0;k<2;k++)
16                 for(l=0;l<2;l++)
17                     if(A[i]+B[j]+C[k]+D[l]==0)
18                         count++;
19
20     printf("Number of tuples = %d",count);
21
22     return 0;
23 }
```

Output

Number of tuples = 2

=== Code Execution Successful ===

11. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.



The screenshot shows a C++ IDE with a file named `main.c`. The code implements a sorting function `sort` using a nested loop algorithm. It also defines a function `medianOfMedians` that calls `sort` and returns the k -th element of the sorted array. The `main` function initializes an array `a = {12, 3, 5, 7, 19}`, sets `n = 5` and `k = 2`, and prints the result. The output window shows the result `K-th smallest element = 5` and a success message.

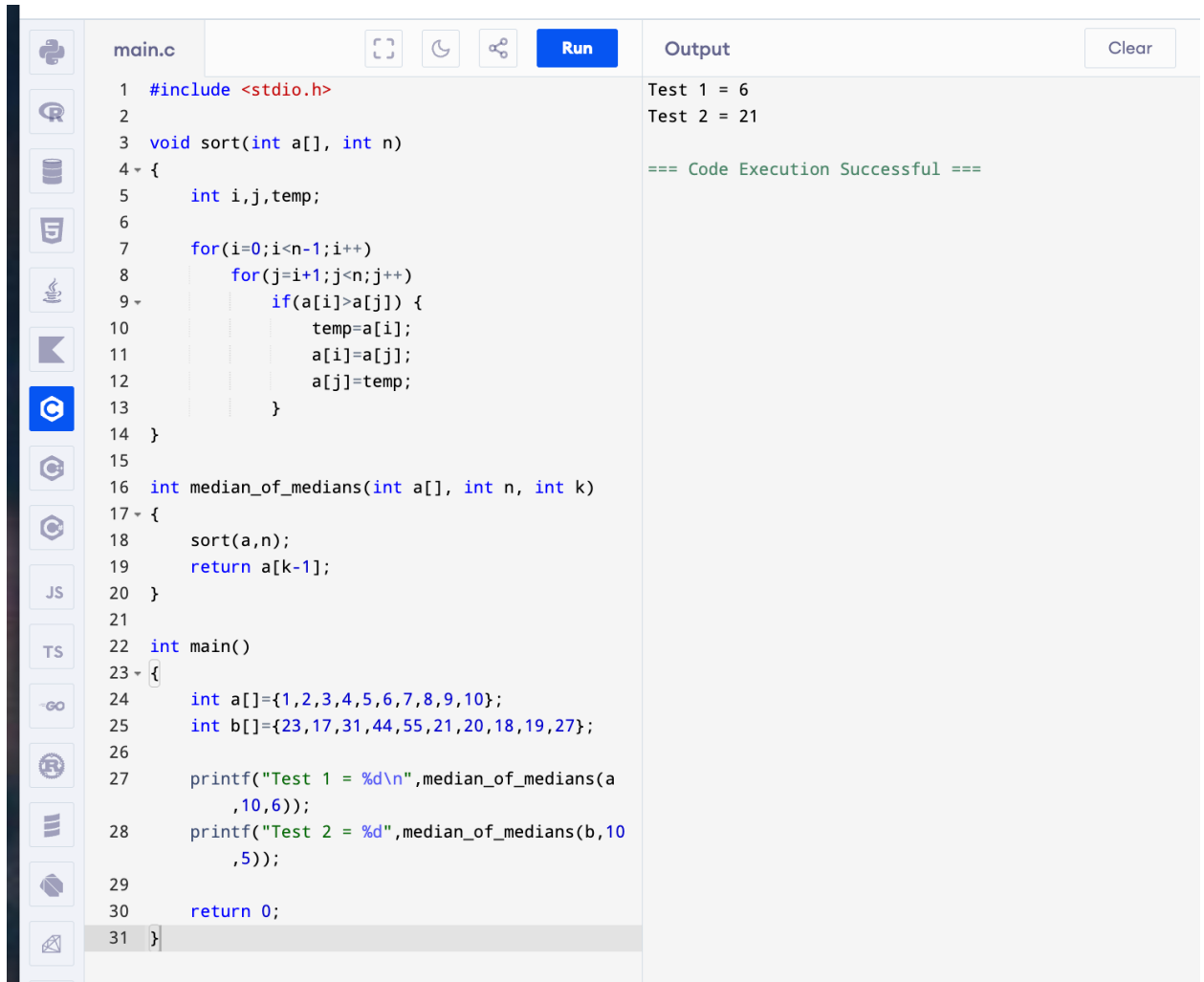
```
1  #include <stdio.h>
2
3  void sort(int a[], int n)
4  {
5      int i,j,temp;
6
7      for(i=0;i<n-1;i++)
8          for(j=i+1;j<n;j++)
9              if(a[i]>a[j]) {
10                 temp=a[i];
11                 a[i]=a[j];
12                 a[j]=temp;
13             }
14 }
15
16 int medianOfMedians(int a[], int n, int k)
17 {
18     sort(a,n);
19     return a[k-1];
20 }
21
22 int main()
23 {
24     int a[]={12,3,5,7,19};
25     int n=5,k=2;
26
27     printf("K-th smallest element = %d",
28           medianOfMedians(a,n,k));
29
30     return 0;
31 }
```

Output

K-th smallest element = 5

=== Code Execution Successful ===

12. To Implement a function `median_of_medians(arr, k)` that takes an unsorted array `arr` and an integer `k`, and returns the k -th smallest element in the array.



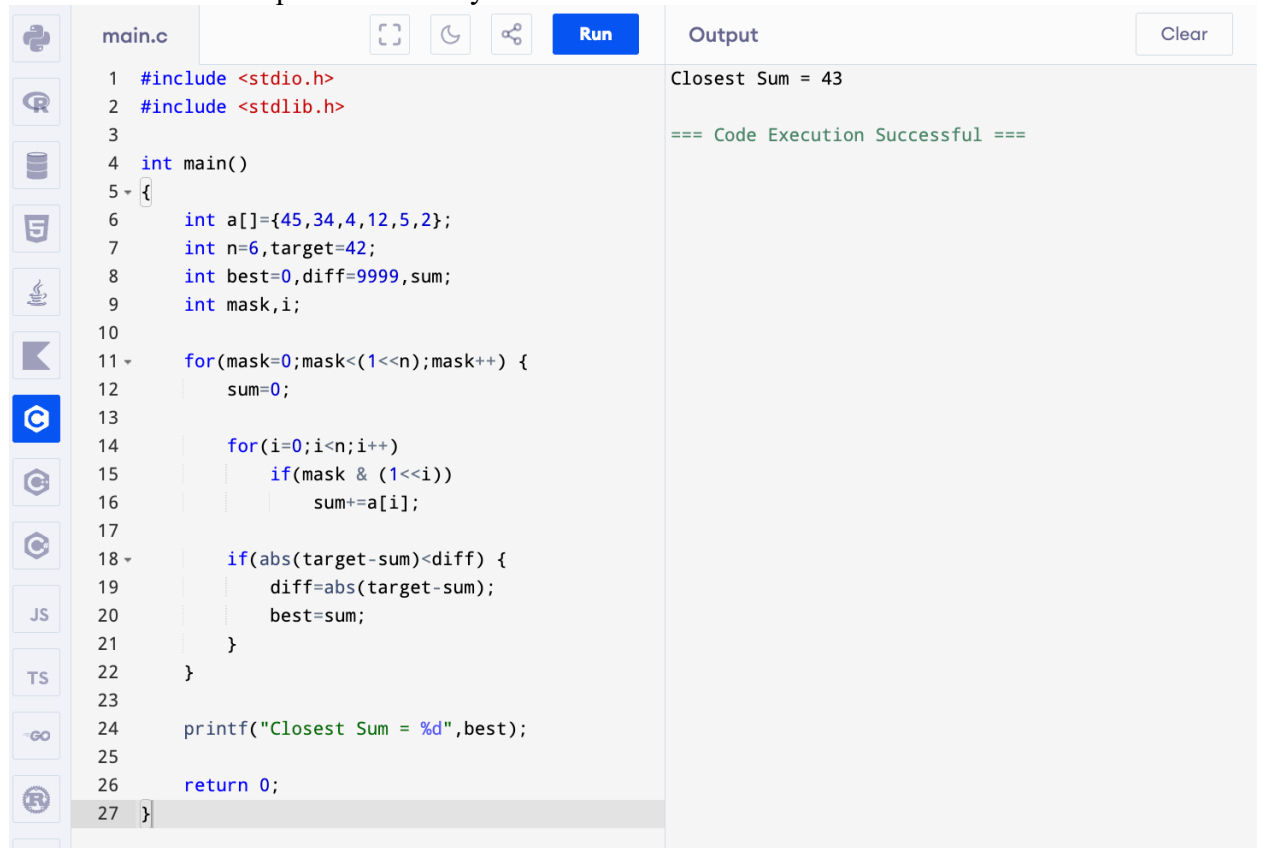
```
main.c
1  #include <stdio.h>
2
3  void sort(int a[], int n)
4  {
5      int i,j,temp;
6
7      for(i=0;i<n-1;i++)
8          for(j=i+1;j<n;j++)
9              if(a[i]>a[j]) {
10                 temp=a[i];
11                 a[i]=a[j];
12                 a[j]=temp;
13             }
14 }
15
16 int median_of_medians(int a[], int n, int k)
17 {
18     sort(a,n);
19     return a[k-1];
20 }
21
22 int main()
23 {
24     int a[]={1,2,3,4,5,6,7,8,9,10};
25     int b[]={23,17,31,44,55,21,20,18,19,27};
26
27     printf("Test 1 = %d\n",median_of_medians(a
28         ,10,6));
29     printf("Test 2 = %d",median_of_medians(b,10
30         ,5));
31     return 0;
32 }
```

Output

Test 1 = 6
Test 2 = 21
=== Code Execution Successful ===

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet

in the Middle technique to efficiently find this subset.



```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int a[]={45,34,4,12,5,2};
7     int n=6,target=42;
8     int best=0,diff=9999,sum;
9     int mask,i;
10
11     for(mask=0;mask<=(1<<n);mask++) {
12         sum=0;
13
14         for(i=0;i<n;i++)
15             if(mask & (1<<i))
16                 sum+=a[i];
17
18         if(abs(target-sum)<diff) {
19             diff=abs(target-sum);
20             best=sum;
21         }
22     }
23
24     printf("Closest Sum = %d",best);
25
26     return 0;
27 }
```

Output

Closest Sum = 43

=== Code Execution Successful ===

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E , determine if there is any subset that sums exactly to E . Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E , otherwise return false.

[illegible]

15 Given two 2×2 Matrices A and B

main.c		Run	Output	Clear
1	#include <stdio.h>		34 22	
2			38 34	
3	int main()			
4	{		=== Code Execution Successful ===	
5	int A[2][2]={1,7},{3,5};			
6	int B[2][2]={6,8},{4,2};			
7	int C[2][2];			
8				
9	int M1=(A[0][0]+A[1][1])*(B[0][0]+B[1][1]);			
10	int M2=(A[1][0]+A[1][1])*B[0][0];			
11	int M3=A[0][0]*(B[0][1]-B[1][1]);			
12	int M4=A[1][1]*(B[1][0]-B[0][0]);			
13	int M5=(A[0][0]+A[0][1])*B[1][1];			
14	int M6=(A[1][0]-A[0][0])*(B[0][0]+B[0][1]);			
15	int M7=(A[0][1]-A[1][1])*(B[1][0]+B[1][1]);			
16				
17	C[0][0]=M1+M4-M5+M7;			
18	C[0][1]=M3+M5;			
19	C[1][0]=M2+M4;			
20	C[1][1]=M1-M2+M3+M6;			
21				
22	printf("%d %d\n",C[0][0],C[0][1]);			
23	printf("%d %d",C[1][0],C[1][1]);			
24				
25	return 0;			
26	}			

16 Given two integers $X=1234$ and $Y=5678$: Use the Karatsuba algorithm to compute the product $Z=X \times Y$

main.c

Run

```
1 #include <stdio.h>
2
3 long long karatsuba(long long x, long long y)
4 {
5     long long a,b,c,d,p=1;
6     long long z0,z1,z2;
7
8     if(x<10 || y<10)
9         return x*y;
10
11     while(x/p>=10)
12         p*=10;
13
14     a=x/p;
15     b=x%p;
16     c=y/p;
17     d=y%p;
18
19     z0=karatsuba(b,d);
20     z2=karatsuba(a,c);
21     z1=karatsuba(a+b,c+d)-z2-z0;
22
23     return z2*p*p+z1*p+z0;
24 }
25
26 int main()
27 {
28     long long x=1234,y=5678;
29
30     printf("Z = %lld",karatsuba(x,y));
31
32     return 0;
33 }
```

Output

Clear

Z = 7006652

=== Code Execution Successful ===